



UNIVERSITÀ DEGLI STUDI DI MILANO BICOCCA

DIPARTIMENTO DI INFORMATICA, SISTEMISTICA E
TELECOMUNICAZIONE
CORSO DI LAUREA MAGISTRALE IN
INFORMATICA

Relazione di Progetto Libreria per Sistemi Lineari

Studenti:

Acquati Luca
Matricola 934996

Jamil Muhammad Qasim
Matricola 932972

Anno accademico:

2024/2025

Indice

Capitolo 1. Introduzione	1
Capitolo 2. Libreria	3
2.1 Struttura del progetto	3
2.2 Solutori implementati	3
2.2.1 Metodo di Jacobi	3
2.2.2 Metodo di Gauß-Seidel	4
2.2.3 Metodo del Gradiente	4
2.2.4 Metodo del Gradiente Coniugato	4
Capitolo 3. Risultati ottenuti	5
3.1 Risultati per la matrice spa1	5
3.1.1 Risultati Numerici	5
3.1.2 Confronto Grafico	7
3.2 Risultati per la matrice spa2	8
3.2.1 Risultati Numerici	8
3.2.2 Confronto Grafico	9
3.3 Risultati per la matrice vem1	10
3.3.1 Risultati Numerici	10
3.3.2 Confronto Grafico	11
3.4 Risultati per la matrice vem2	12
3.4.1 Risultati Numerici	12
3.4.2 Confronto Grafico	13
Capitolo 4. Conclusioni	15
4.1 Efficienza e numero di iterazioni	15
4.2 Tempi di esecuzione	15
4.3 Accuratezza e convergenza	15
4.4 Considerazioni finali	16

Capitolo 1

Introduzione

L'obiettivo del progetto è lo sviluppo di una mini libreria per la risoluzione di sistemi lineari tramite metodi iterativi, con particolare attenzione al caso di matrici simmetriche e definite positive. Questo lavoro si inserisce nell'ambito del corso di Metodi del Calcolo Scientifico per l'anno accademico 2024-2025 e ha come finalità l'approfondimento degli algoritmi numerici per la risoluzione di sistemi di grandi dimensioni, spesso caratterizzati da matrici sparse.

Nel dettaglio, la libreria implementa i seguenti quattro solutori iterativi:

- Metodo di Jacobi
- Metodo di Gauß-Seidel
- Metodo del Gradiente
- Metodo del Gradiente Coniugato

Come linguaggio di programmazioni è stato scelto Python e per la gestione di matrici e vettori si è fatto uso di una libreria open-source, limitandosi esclusivamente alle strutture dati e alle operazioni elementari, senza ricorrere a solutori già implementati.

La libreria sviluppata è in grado di:

- Caricare matrici simmetriche e definite positive in formato `.mtx`
- Applicare ciascun metodo iterativo partendo da un vettore iniziale nullo
- Arrestarsi al raggiungimento di una tolleranza specificata dall'utente o al superamento di un numero massimo di iterazioni
- Restituire in output l'errore relativo, il numero di iterazioni richieste e il tempo di calcolo per ciascun metodo

Il progetto è stato validato tramite una serie di test sulle matrici fornite (`spa1.mtx`, `spa2.mtx`, `vem1.mtx`, `vem2.mtx`), seguendo la procedura standard di generazione della soluzione esatta e calcolo dell'errore relativo per diverse scelte della tolleranza.

Questa relazione descrive la struttura della libreria, la metodologia di validazione adottata, i risultati ottenuti e le considerazioni derivanti dai confronti tra i diversi metodi.

Capitolo 2

Libreria

La libreria è stata progettata con una struttura modulare e ordinata per garantire una gestione efficiente delle matrici, dei solutori e dei risultati ottenuti. Il progetto è organizzato in modo da separare chiaramente le funzionalità di calcolo da quelle di input/output, favorendo la leggibilità e la facilità di manutenzione.

2.1 Struttura del progetto

Il progetto è suddiviso nelle seguenti directory:

- **dati**: contiene le matrici in formato `.mtx` fornite per la validazione dei solutori;
- **library**: contiene l'implementazione dei quattro metodi iterativi richiesti e delle funzioni necessarie alla creazione dei grafici;
- **plots**: utilizzata per salvare i grafici ottenuti in sottocartelle con timestamp dell'esecuzione;
- **main.py**: file principale per l'esecuzione del progetto.

2.2 Solutori implementati

La libreria sviluppata include quattro solutori iterativi per la risoluzione di sistemi lineari con matrici simmetriche e definite positive. Ogni metodo è stato implementato in un modulo separato all'interno della cartella `library`.

2.2.1 Metodo di Jacobi

Il metodo di Jacobi è un algoritmo iterativo che calcola la soluzione di ogni componente del vettore incognita basandosi esclusivamente sui valori dell'iterazione precedente. Il metodo è semplice da implementare e ben parallelizzabile, ma generalmente presenta una velocità di convergenza più lenta rispetto ad altri metodi.

2.2.2 Metodo di Gauß-Seidel

Il metodo di Gauß-Seidel migliora il metodo di Jacobi utilizzando i valori aggiornati delle componenti man mano che vengono calcolate all'interno della stessa iterazione. Questo consente, in genere, una convergenza più rapida rispetto al metodo di Jacobi, soprattutto per matrici ben condizionate.

2.2.3 Metodo del Gradiente

Il metodo del Gradiente (o metodo del gradiente semplice) sfrutta la discesa lungo la direzione del gradiente per minimizzare l'errore residuo ad ogni iterazione. È efficace per matrici simmetriche e definite positive, ma può richiedere un gran numero di iterazioni per ottenere una soluzione sufficientemente precisa.

2.2.4 Metodo del Gradiente Coniugato

Il metodo del Gradiente Coniugato rappresenta l'approccio più efficiente tra quelli implementati per il tipo di problemi trattati. Rispetto al metodo del gradiente semplice, questo algoritmo genera direzioni di ricerca coniugate che garantiscono una più rapida riduzione dell'errore, spesso con un numero molto minore di iterazioni.

Capitolo 3

Risultati ottenuti

I test sono stati eseguiti utilizzando le quattro matrici fornite (`spa1.mtx`, `spa2.mtx`, `vem1.mtx`, `vem2.mtx`) e le seguenti tolleranze $\text{tol} = [10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}]$. Per ogni matrice sono stati applicati i quattro metodi iterativi: Jacobi, Gauss-Seidel, Gradiente e Gradiente Coniugato e per ciascun metodo sono stati misurati:

- Il numero di iterazioni necessarie per raggiungere la tolleranza specificata;
- Il tempo di esecuzione;
- L'errore relativo.

Di seguito sono riportati i risultati ottenuti per ciascuna matrice.

3.1 Risultati per la matrice `spa1`

3.1.1 Risultati Numerici

Tolleranza 10^{-4}

Method	Iterations	Error	Time (s)
Jacobi	115	1.77e-03	0.0788
Gauss-Seidel	9	1.82e-02	0.0756
Gradient	143	3.46e-02	0.0369
Conjugate Gradient	49	2.08e-02	0.0110

Tolleranza 10^{-6}

Method	Iterations	Error	Time (s)
Jacobi	181	1.80e-05	0.1630
Gauss-Seidel	17	1.30e-04	0.1264
Gradient	3577	9.68e-04	1.2502
Conjugate Gradient	134	2.55e-05	0.0475

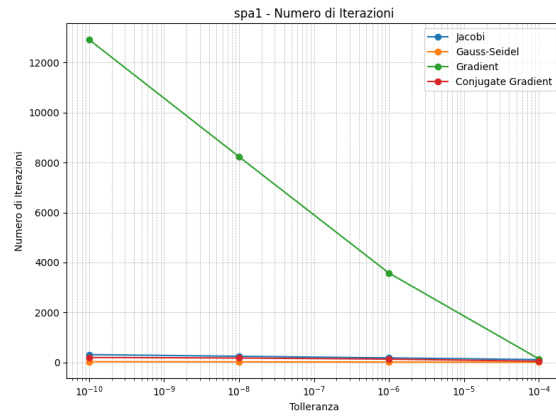
Tolleranza 10^{-8}

Method	Iterations	Error	Time (s)
Jacobi	247	1.82e-07	0.1946
Gauss-Seidel	24	1.71e-06	0.1560
Gradient	8233	9.82e-06	3.7484
Conjugate Gradient	177	1.32e-07	0.0759

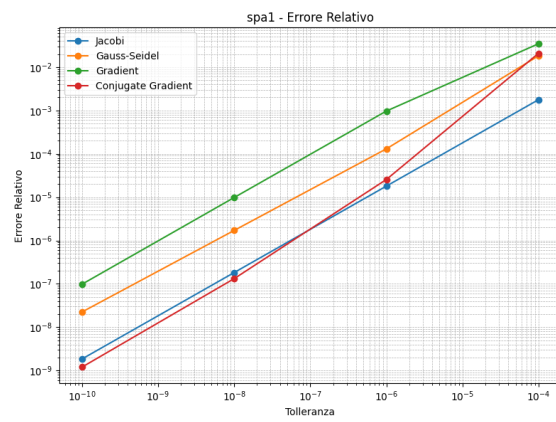
Tolleranza 10^{-10}

Method	Iterations	Error	Time (s)
Jacobi	313	1.85e-09	0.2684
Gauss-Seidel	31	2.25e-08	0.2334
Gradient	12919	9.82e-08	4.9202
Conjugate Gradient	200	1.20e-09	0.0732

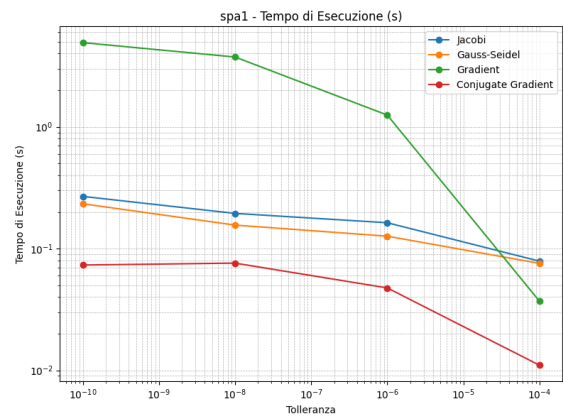
3.1.2 Confronto Grafico



(a) Iterazioni



(b) Errore



(c) Tempo

3.2 Risultati per la matrice spa2

3.2.1 Risultati Numerici

Tolleranza 10^{-4}

Method	Iterations	Error	Time (s)
Jacobi	36	1.77e-03	0.5462
Gauss-Seidel	5	2.60e-03	0.2053
Gradient	161	1.81e-02	0.6155
Conjugate Gradient	42	9.82e-03	0.1596

Tolleranza 10^{-6}

Method	Iterations	Error	Time (s)
Jacobi	57	1.67e-05	0.5418
Gauss-Seidel	8	5.14e-05	0.3479
Gradient	1949	6.69e-04	8.1685
Conjugate Gradient	122	1.20e-04	0.5969

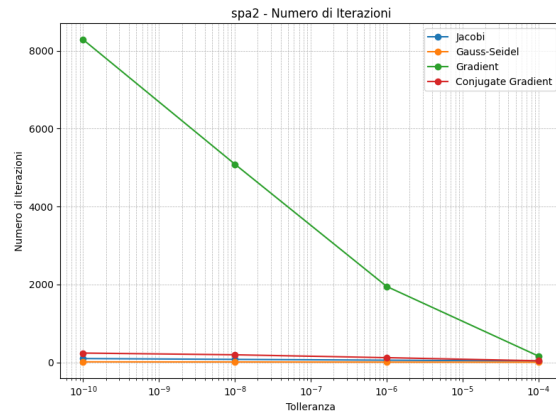
Tolleranza 10^{-8}

Method	Iterations	Error	Time (s)
Jacobi	78	1.57e-07	0.7984
Gauss-Seidel	12	2.79e-07	0.4533
Gradient	5087	6.87e-06	25.4162
Conjugate Gradient	196	5.59e-07	1.0760

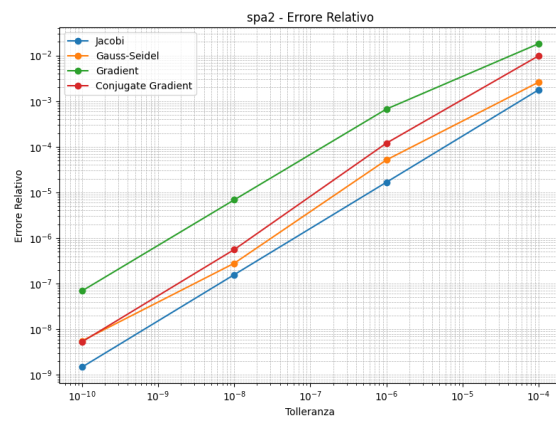
Tolleranza 10^{-10}

Method	Iterations	Error	Time (s)
Jacobi	99	1.48e-09	1.1927
Gauss-Seidel	15	5.57e-09	0.6423
Gradient	8285	6.94e-08	48.6039
Conjugate Gradient	240	5.32e-09	1.3309

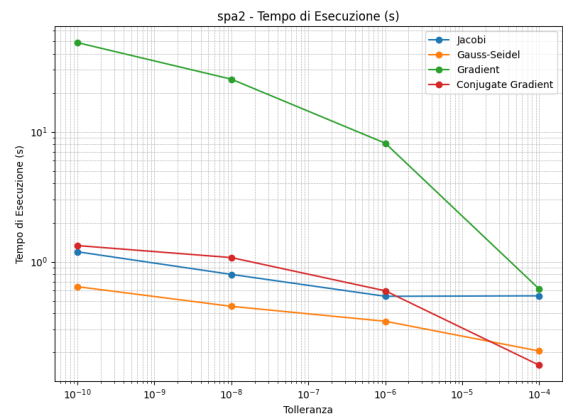
3.2.2 Confronto Grafico



(a) Iterazioni



(b) Errore



(c) Tempo

3.3 Risultati per la matrice vem1

3.3.1 Risultati Numerici

Tolleranza 10^{-4}

Method	Iterations	Error	Time (s)
Jacobi	1314	3.54e-03	4.3364
Gauss-Seidel	659	3.51e-03	9.9516
Gradient	890	2.70e-03	1.3353
Conjugate Gradient	38	4.08e-05	0.0527

Tolleranza 10^{-6}

Method	Iterations	Error	Time (s)
Jacobi	2433	3.54e-05	7.9643
Gauss-Seidel	1218	3.53e-05	17.5357
Gradient	1612	2.71e-05	2.3385
Conjugate Gradient	45	3.73e-07	0.0652

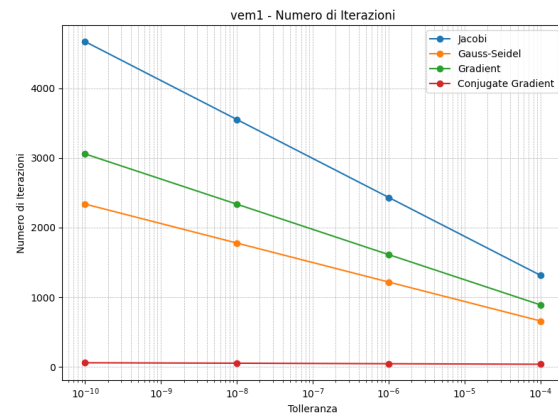
Tolleranza 10^{-8}

Method	Iterations	Error	Time (s)
Jacobi	3552	3.54e-07	12.1987
Gauss-Seidel	1778	3.52e-07	25.3576
Gradient	2336	2.70e-07	3.3838
Conjugate Gradient	53	2.83e-09	0.0742

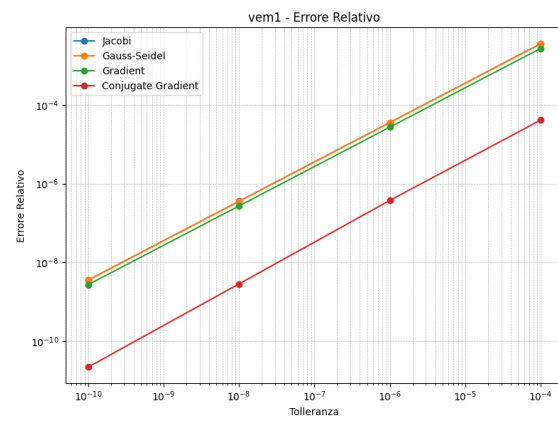
Tolleranza 10^{-10}

Method	Iterations	Error	Time (s)
Jacobi	4671	3.54e-09	14.2767
Gauss-Seidel	2338	3.51e-09	32.9100
Gradient	3058	2.71e-09	4.3177
Conjugate Gradient	59	2.19e-11	0.0909

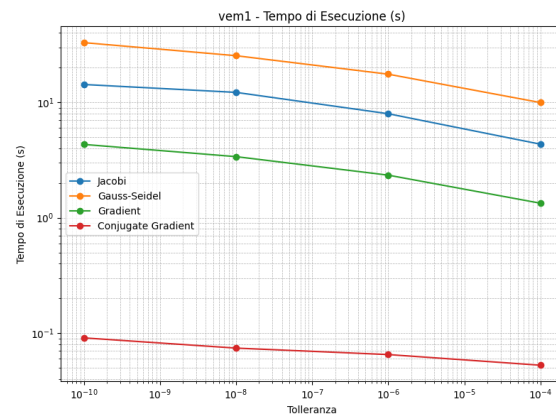
3.3.2 Confronto Grafico



(a) Iterazioni



(b) Errore



(c) Tempo

3.4 Risultati per la matrice vem2

3.4.1 Risultati Numerici

Tolleranza 10^{-4}

Method	Iterations	Error	Time (s)
Jacobi	1927	4.97e-03	18.6796
Gauss-Seidel	965	4.95e-03	32.4025
Gradient	1308	3.81e-03	5.0909
Conjugate Gradient	47	5.73e-05	0.1805

Tolleranza 10^{-6}

Method	Iterations	Error	Time (s)
Jacobi	3676	4.97e-05	28.5240
Gauss-Seidel	1840	4.94e-05	45.8861
Gradient	2438	3.79e-05	7.8939
Conjugate Gradient	56	4.74e-07	0.2170

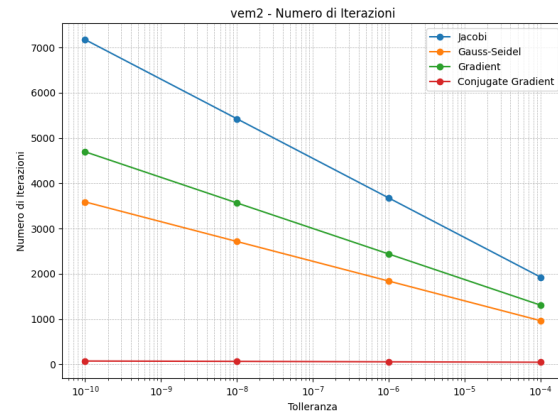
Tolleranza 10^{-8}

Method	Iterations	Error	Time (s)
Jacobi	5425	4.97e-07	39.5954
Gauss-Seidel	2714	4.96e-07	65.3898
Gradient	3566	3.81e-07	10.8606
Conjugate Gradient	66	4.30e-09	0.2113

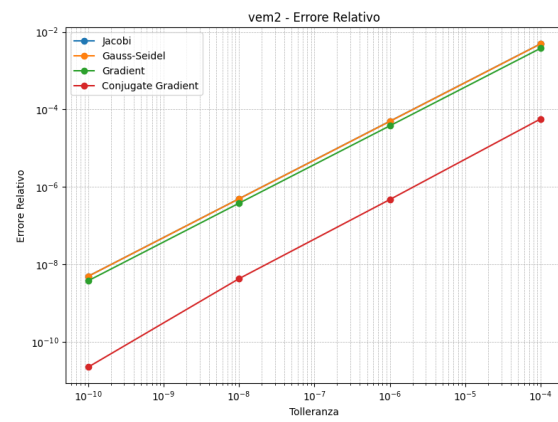
Tolleranza 10^{-10}

Method	Iterations	Error	Time (s)
Jacobi	7174	4.96e-09	46.4217
Gauss-Seidel	3589	4.95e-09	78.7828
Gradient	4696	3.80e-09	13.8764
Conjugate Gradient	74	2.25e-11	0.2367

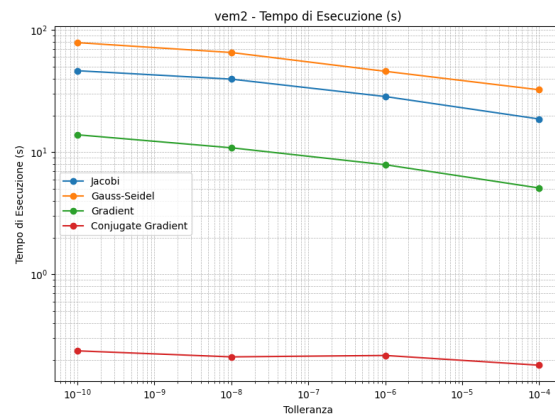
3.4.2 Confronto Grafico



(a) Iterazioni



(b) Errore



(c) Tempo

Capitolo 4

Conclusioni

4.1 Efficienza e numero di iterazioni

Il Gradiente Coniugato si dimostra il più efficiente in termini di iterazioni su tutte le matrici, mantenendo un numero relativamente basso anche con tolleranze molto stringenti.

- Su `spa1.mtx` e `spa2.mtx`, matrici più piccole e sparse, il numero di iterazioni per il Gradiente Coniugato rimane contenuto;
- Per le matrici `vem1.mtx` e `vem2.mtx`, più grandi e complesse, il numero di iterazioni cresce ma resta sempre inferiore rispetto agli altri metodi, confermando la scalabilità del metodo.

Al contrario, il metodo del Gradiente richiede un numero di iterazioni molto alto su tutte le matrici, in particolare su quelle più grandi, indicando una minore efficienza su problemi più complessi.

4.2 Tempi di esecuzione

I tempi di calcolo rispecchiano la complessità delle matrici:

- Su `spa1.mtx` e `spa2.mtx` i tempi di esecuzione sono contenuti per tutti i metodi, con il Gradiente Coniugato che resta sempre molto più veloce rispetto agli altri;
- Su `vem1.mtx` e `vem2.mtx`, i tempi per Jacobi e Gauss-Seidel aumentano sensibilmente.

Anche il metodo del Gradiente ha tempi elevati sulle matrici grandi, rendendolo poco pratico in questi casi. Il Gradiente Coniugato, invece, mantiene tempi contenuti, con un incremento proporzionale alla dimensione ma sempre competitivo.

4.3 Accuratezza e convergenza

Tutti i metodi riescono a raggiungere l'accuratezza richiesta per ogni matrice, ma con differenze di efficienza:

- Per le matrici `spa1.mtx` e `spa2.mtx`, Jacobi e Gauss-Seidel raggiungono errori molto bassi in un numero limitato di iterazioni, anche se con più tempo rispetto al Gradiente Coniugato;
- Su `vem1.mtx` e `vem2.mtx`, il Gradiente Coniugato riesce a ottenere errori molto piccoli in tempi relativamente brevi, mentre Jacobi e Gauss-Seidel necessitano di molte più iterazioni e di conseguenza tempo per ottenere una precisione simile.
- Il metodo del Gradiente raggiunge errori bassi, ma con un costo iterativo e temporale molto più alto, soprattutto nelle matrici più grandi.

4.4 Considerazioni finali

Analizzando i risultati matrice per matrice, si nota che il Gradiente Coniugato è il metodo più robusto e scalabile, particolarmente indicato per matrici grandi e complesse come `vem1.mtx` e `vem2.mtx`, dove gli altri metodi mostrano limiti in termini di tempi e iterazioni. I metodi di Jacobi e Gauss-Seidel possono essere accettabili e competitivi su matrici di dimensioni più ridotte e con tolleranze meno stringenti, come per `spa1.mtx` e `spa2.mtx`, ma diventano meno efficienti con matrici grandi. Il metodo del Gradiente, pur affidabile, risulta poco conveniente nelle applicazioni pratiche su matrici di grandi dimensioni a causa dell'alto costo computazionale.